COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 22, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Johrel Louie T. Silao	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _3.1_ Creating a Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#include <iostream>
#include "singly_ll.h"

                                        // === Creation of Nodes "JOLO" ====

int main(){

    // Creating a Node Stored in the Stack

    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //Allocate Memory in the Heap

    head = new SingleList<char>();
    second = new SingleList<char>();
    third = new SingleList<char>();
    fourth = new SingleList<char>();

    //Storing a Value Inside the Node

    head->data = 'J';
    second->data = 'O';
    third->data = 'L';
    fourth->data = 'O';
```

```cpp
    //Linking the Nodes

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout << "Testing of Traversal: \n";
    ListTraversal(head);

     std::cout << "Testing of sll_insert_head: \n";
    sll_insert_head('Q', &head);
    ListTraversal(head);

    std::cout << "Testing of sll_general_insert: \n";
    sll_general_insert('X', head->next->next);
    ListTraversal(head);

    std::cout<<"Testing of sll_insert_end: \n";
    sll_insert_end('U', &head);
    ListTraversal(head);

    std::cout<<"Testing of deleting the node: \n";
    sll_Delete_Node('U', &head);
    ListTraversal(head);
    sll_Delete_Node('Q', &head);
    ListTraversal(head);
    sll_Delete_Node('X', &head);
    ListTraversal(head);


    std::cout<<"Deleting all nodes: \n";
    sll_Deleting_List(&head);
    ListTraversal(head);

    return 0;

}
```

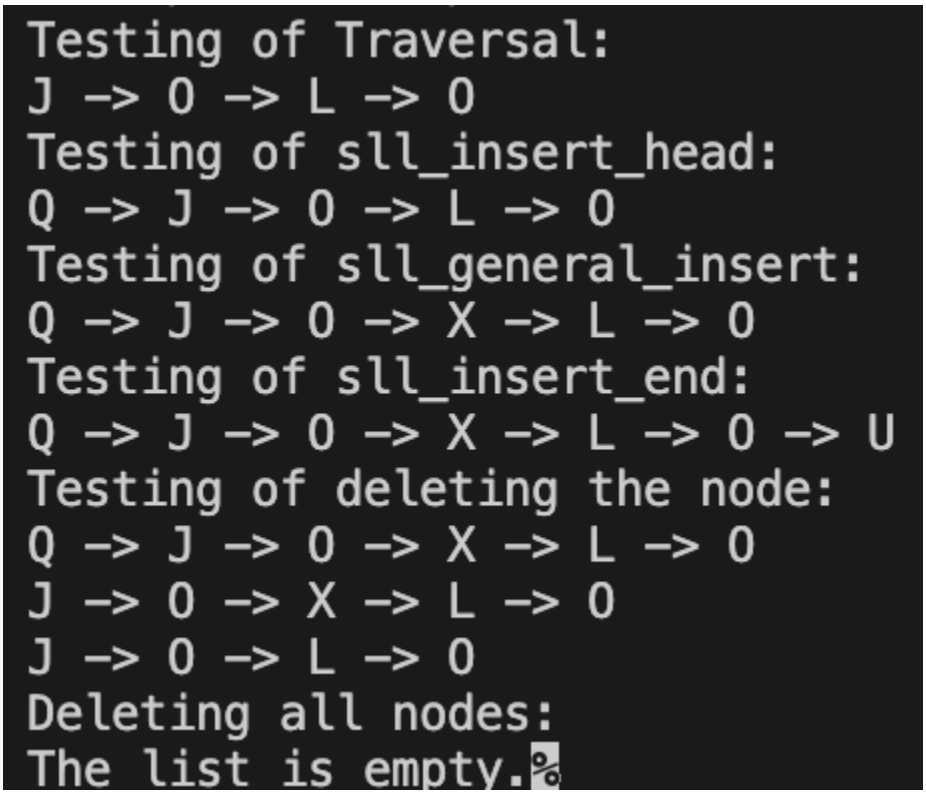
```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

// NODE CREATION

template <typename T>
class SingleList{
    public:
        T data;
        SingleList<T>* next = nullptr;
};

template <typename T>
void ListTraversal(SingleList<T>* head){
    if(head == nullptr){
        std::cout<<"The list is empty.";
        return;
    }
    while (head != nullptr){
        // Print data of N

        std::cout << head->data;

        if(head->next != nullptr){
            std::cout << " -> ";
        }

        // Go to the next node

        head = head->next;
    }

    std::cout << std::endl;

}
```

```cpp
template <typename T>
void sll_insert_head(T newData, SingleList<T>** currentHead){
    // 1. Allocate memory for the new node

    SingleList<T>* newNode = new SingleList<T>;

    // 2. Put our data into the new node

    newNode->data = newData;

    // 3. Set the next of the new node to point to the previous head

    newNode->next = *currentHead;

    // 4. Reset head to point to the new node

    *currentHead = newNode;

}
```

```cpp
template <typename T>
// 1. Check if it is the head node(previous node is null)

void sll_general_insert(T newData, SingleList<T>* prevNode){
    // 2. If null, print "Previous node cannot be nullptr" and return

    if(prevNode == nullptr){
        std::cout <<"Previous node cannot be nullptr" << std::endl;
        return;
    }
    // 3. Allocate a new node

    SingleList<T>* newNode = new SingleList<T>;
   // 4. Store data in the new node

   newNode->data = newData;
   // 5. Point to the new node to the node previous node was pointing to

   newNode->next = prevNode->next;
    // 6. Point previous node to the new node

    prevNode-> next = newNode;
}
```

```cpp
template <typename T>
void sll_insert_end(T newData, SingleList<T>** head){
    // 1. Allocate new node

    SingleList<T>* newNode = new SingleList<T>;
    // 2. Dereference to the head node

    SingleList<T>* currentNode = *head;
    // 3. Store data in new node

    newNode->data = newData;
    // 4. Point to null

    newNode-> next = nullptr;
    // 5. Traverse the list until pointed to null

    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }
    // 6. Point the next of the current node to the new node

    currentNode->next = newNode;
}

```

```
118     while(currNode != nullptr && currNode->data != findData){
119         prevNode =currNode;
120         currNode = prevNode->next;
121     }
122     // If data not found currNode == nullptr in the while loop:
123 
124     if(currNode == nullptr){
125         std::cout<<"The data is not found \n"<<std::endl;
126     }
127     // If data was found
128 
129     if(prevNode == nullptr){
130         *head = currNode->next;
131     }else{
132         prevNode->next = currNode->next;
133     }
134     delete currNode;
135 }
136 
137 template <typename T>
138 void sll_Deleting_List(SingleList<T> **head){
139     SingleList<T>* current = *head;
140 
141     while(current != nullptr){
142 
143         SingleList<T>* temp = current;
144         current = current->next;
145         delete temp;
146     }
147 
148     *head = nullptr;
149 }
150 #endif
```



Conclusion:

In this activity, it shows all the operations of a Singly Linked List such as, the list initialization and traversal, dynamic insertions, and node deletion and cleanup. The nodes were initiated on the heap and traversed sequentially to show the character sequence and proceeded to test that the head, middle and the end are corresponding to the connections. Additionally, some of the specific nodes were isolated and the sll_Deleting_List clears all the remaining nodes, which then returns the list to an empty list. I was left behind at the class and finishing this activity, so I collaborated with some of my classmates and friends on how I can solve my problem. In the end, I was able to finish the activity and show the expected output of the program.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.